

Documentação do JSBase 2.1

1. Introdução

O JSBase é uma linguagem de programação simplificada, desenvolvida com o propósito educacional de introduzir estudantes aos conceitos fundamentais da programação. Inspirado na sintaxe do JavaScript, o JSBase oferece uma curva de aprendizado suave, permitindo que os alunos se concentrem na lógica de programação sem se preocuparem com as complexidades de linguagens mais avançadas. A versão 2.1 representa uma evolução contínua, incorporando novas funcionalidades e aprimoramentos que o tornam uma ferramenta ainda mais robusta e completa para o ensino de programação no SENAI.

2. Funcionalidades da Versão 2.1

A versão 2.1 do JSBase foi aprimorada para oferecer uma experiência de desenvolvimento mais fluida e educativa. As principais melhorias e funcionalidades suportadas incluem:

- **✓ Tipos de Variáveis:** `numero`, `texto`, `booleano`.
- **✓ Atribuição:** Comando `recebe` para atribuição de valores e reatribuição.
- **✓ Saída de Dados:** Comando `mostrar` com suporte a múltiplos argumentos.
- **✓ Condicionais:** Estruturas `se`, `senão` e `fimse`.
- **✓ Laços de Repetição:** `enquanto` (equivalente a `while`) e `para` (equivalente a `for`).
- **✓ Validação Estrutural:** Verificação de blocos (`se`, `enquanto`, `para`) para garantir que sejam corretamente abertos e fechados.
- **✓ Sistema de Erros Aprimorado:** Mensagens de erro claras com indicação da linha onde o problema ocorreu.

- **✓ Case Insensitive:** A linguagem agora é insensível a maiúsculas e minúsculas para palavras-chave, proporcionando maior flexibilidade na escrita do código.
- **✓ Robustez no Processamento:** Regex otimizadas para maior flexibilidade de espaçamento e posicionamento de comandos.
- **✓ Editor Aprimorado:** Sincronização de scroll e destaque de sintaxe para palavras reservadas, strings, números e comentários.

2.1. Laço de Repetição para

O laço `para` foi introduzido para permitir iterações controladas por um contador, similar ao `for` do JavaScript. Ele oferece uma maneira concisa de executar um bloco de código um número específico de vezes.

2.2. Validação Estrutural de Blocos

Para auxiliar na escrita de código correto, o JSBase 2.1 agora inclui uma validação estrutural que verifica se os blocos de código (`se` , `enquanto` , `para`) são devidamente abertos e fechados. Erros como um `fimse` sem um `se` correspondente são detectados e reportados com a linha exata do problema.

2.3. Case Insensitive

As palavras-chave da linguagem (ex: `numero` , `recebe` , `se` , `mostrar`) agora podem ser escritas em qualquer combinação de maiúsculas e minúsculas (ex: `Numero` , `RECEBE` , `Se`), tornando a linguagem mais amigável e perdoando pequenos erros de digitação relacionados a capitalização.

3. Sintaxe da Linguagem JSBase 2.1

Esta seção detalha a sintaxe dos comandos disponíveis no JSBase 2.1 e sua correspondência com o JavaScript. Lembre-se que as palavras-chave são **case insensitive**.

3.1. Declaração e Atribuição de Variáveis

O JSBase suporta três tipos de variáveis: `numero`, `texto` e `booleano`. A atribuição de valores é feita com o comando `recebe`.

Comando JSBase	Equivalente JavaScript	Descrição
<code>numero minhaIdade recebe 25</code>	<code>let minhaIdade = 25;</code>	Declara uma variável numérica e atribui um valor.
<code>texto meuNome recebe "João"</code>	<code>let meuNome = "João";</code>	Declara uma variável de texto e atribui um valor.
<code>booleano estaAtivo recebe verdadeiro</code>	<code>let estaAtivo = true;</code>	Declara uma variável booleana e atribui um valor.
<code>minhaIdade recebe minhaIdade + 1</code>	<code>minhaIdade = minhaIdade + 1;</code>	Atribui um novo valor a uma variável existente.

Exemplo:

```
NUMERO idade recebe 10
texto nome recebe "Maria"
booleano ativo recebe verdadeiro

idade RECEBE (idade + 5) * 2
mostrar idade
```

3.2. Saída de Dados (`mostrar`)

O comando `mostrar` é utilizado para exibir informações no console de saída.

Comando JSBase	Equivalente JavaScript	Descrição
<code>mostrar "Olá, mundo!"</code>	<code>console.log("Olá, mundo!");</code>	Exibe uma string literal.
<code>mostrar minhaVariavel</code>	<code>console.log(minhaVariavel);</code>	Exibe o valor de uma variável.
<code>mostrar "Nome:", meuNome</code>	<code>console.log("Nome:", meuNome);</code>	Exibe múltiplos valores separados por vírgula.

Exemplo:

```
texto saudacao recebe "Bem-vindo ao JSBase!"
mostrar saudacao
mostrar "O resultado é:", 10 + 20
```

3.3. Condicionais (se , senão , fimse)

Permitem a execução condicional de blocos de código.

Comando JSBase	Equivalente JavaScript	Descrição
<code>se (idade maior que 18)</code>	<code>if (idade > 18) {</code>	Inicia um bloco condicional <code>if</code> .
<code>senão</code>	<code>} else {</code>	Inicia um bloco <code>else</code> (opcional).
<code>fimse</code>	<code>}</code>	Encerra o bloco condicional.

Exemplo:

```
numero idade recebe 20
se (idade maior que 18)
    mostrar "Maior de idade"
senão
    mostrar "Menor de idade"
fimse
```

3.4. Laços de Repetição (enquanto , para , fimenquanto , fimpara)

Executam um bloco de código repetidamente enquanto uma condição for verdadeira (enquanto) ou por um número determinado de vezes (para).

Comando JSBase	Equivalente JavaScript	Descrição
enquanto (contador menor que 5)	<pre>while (contador < 5) {</pre>	Inicia um laço de repetição while .
fimenquanto	<pre>}</pre>	Encerra o laço enquanto .
para (i recebe 0; i menor que 5; i recebe i + 1)	<pre>for (let i = 0; i < 5; i++) {</pre>	Inicia um laço de repetição for .
fimpara	<pre>}</pre>	Encerra o laço para .

Exemplo enquanto :

```
numero contador recebe 0
enquanto (contador menor que 3)
    mostrar "Contador:", contador
    contador recebe contador + 1
fimenquanto
```

Exemplo para :

```
para (numero i recebe 0; i menor que 5; i recebe i + 1)
    mostrar "Iteração:", i
fimpara
```

3.5. Comentários (//)

Comentários são linhas de texto ignoradas pelo interpretador, usadas para documentar o código.

Comando JSBase	Equivalente JavaScript	Descrição
// Este é um comentário	// Este é um comentário	Comentário de linha única.

Exemplo:

```
// Este programa calcula a soma de dois números
numero a recebe 10
numero b recebe 20
mostrar a + b // Exibe o resultado
```

3.6. Operadores

O JSBase suporta operadores matemáticos, relacionais e lógicos, que são traduzidos para seus equivalentes em JavaScript.

3.6.1. Operadores Relacionais

Operador JSBase	Equivalente JavaScript	Descrição
maior ou igual	<code>>=</code>	Maior ou igual a.
menor ou igual	<code><=</code>	Menor ou igual a.
maior que	<code>></code>	Maior que.
menor que	<code><</code>	Menor que.
diferente	<code>!=</code>	Diferente de.
igual	<code>==</code>	Igual a.

3.6.2. Operadores Lógicos

Operador JSBase	Equivalente JavaScript	Descrição
e	<code>&&</code>	Operador lógico AND.
ou	<code> </code>	Operador lógico OR.

Exemplo:

```
numero x recebe 10
numero y recebe 5

se (x maior que y e y diferente 0)
    mostrar "Condição verdadeira"
fimse
```

4. Arquitetura Técnica

O JSBase 2.1 opera como um transpiler cliente-side, ou seja, ele converte o código escrito em JSBase para JavaScript diretamente no navegador do usuário antes da execução. A arquitetura é composta por:

- **HTML:** Estrutura da página, incluindo o editor (`textarea`), a área de destaque (`div` com `pre`) e a área de saída.
- **CSS:** Estilização do editor, destaque de sintaxe e layout geral.
- **JavaScript (Core):**
 - **Função `transpilar(codigoJSBase)`** : Esta é a parte central. Ela percorre o código JSBase linha por linha, aplicando expressões regulares para identificar e substituir os comandos JSBase por seus equivalentes em JavaScript. Esta função também gerencia a `pilhaBlocos` para a validação estrutural de `se` , `enquanto` e `para` .
 - **Função `traduzirOperadores(texto)`** : Uma função auxiliar que converte operadores relacionais e lógicos do português para seus símbolos JavaScript.
 - **Função `executar()`** : Captura o código do editor, chama `transpilar()` , e então utiliza `eval()` para executar o JavaScript resultante. Ela também intercepta `console.log` para exibir a saída na área designada e implementa o tratamento de erros aprimorado, incluindo a detecção de erros de validação de blocos.
 - **Função `updateHighlight()`** : Responsável pelo destaque de sintaxe. Ela escapa o HTML do texto do editor e aplica expressões regulares para envolver palavras reservadas, strings, números e comentários com tags `<span>` com classes CSS específicas. A ordem de aplicação das regex é

crucial para evitar que, por exemplo, palavras reservadas dentro de strings sejam destacadas.

- **Sincronização de Scroll:** Listeners de evento garantem que o scroll do `textarea` e do `highlight` estejam sempre sincronizados.

5. Considerações de Segurança

É importante notar que o JSBase 2.1 utiliza a função `eval()` do JavaScript para executar o código transpilado. Em ambientes de produção ou em aplicações que lidam com dados sensíveis de usuários, o uso de `eval()` é **fortemente desencorajado** devido aos riscos de segurança, como a injeção de código malicioso. No entanto, para o contexto educacional do JSBase, onde o objetivo é o aprendizado e o ambiente é controlado (execução local no navegador do aluno), esse risco é considerado aceitável e gerenciável. Para futuras versões ou aplicações mais complexas, alternativas mais seguras, como a construção de uma Abstract Syntax Tree (AST) e um interpretador próprio, seriam recomendadas.

6. Como Usar

1. Abra o arquivo `jsbase_corrigido.html` (ou a versão mais recente do seu projeto) em qualquer navegador web moderno.
2. Digite seu código JSBase na área de texto principal.
3. Clique no botão “Executar Código” para ver o resultado na área de saída abaixo.
4. Observe o destaque de sintaxe em tempo real enquanto digita.
5. Em caso de erros, a mensagem de erro indicará a linha aproximada onde o problema ocorreu, incluindo erros de validação estrutural de blocos.

7. Exemplos de Código

Exemplo 1: Cálculo Simples e Condicional

```
// Calcula a média de duas notas e verifica aprovação
NUMERO nota1 recebe 7
numero nota2 recebe 8.5
numero media recebe (nota1 + nota2) / 2

mostrar "Média:", media

se (media maior ou igual 7)
    mostrar "Aluno Aprovado!"
senão
    mostrar "Aluno Reprovado!"
fimse
```

Exemplo 2: Laço de Repetição enquanto

```
// Exibe números de 1 a 5
numero contador recebe 1
enquanto (contador menor ou igual 5)
    mostrar "Contador:", contador
    contador recebe contador + 1
fimenquanto
mostrar "Fim da contagem."
```

Exemplo 3: Laço de Repetição para

```
// Exibe números de 0 a 4 usando o laço para
para (numero i recebe 0; i menor que 5; i recebe i + 1)
    mostrar "Iteração do laço PARA:", i
fimpara
mostrar "Laço PARA finalizado."
```

Exemplo 4: Variáveis de Texto e Booleano com Case Insensitive

```
TEXTO saudacao recebe "Olá, "  
texto nomeUsuario recebe "Mundo"  
BOOLEANO logado recebe verdadeiro  
  
SE (logado igual verdadeiro)  
    mostrar saudacao + nomeUsuario + "! Você está logado."  
SENÃO  
    mostrar "Por favor, faça login."  
FIMSE
```

Autor: Elcio Julio Sava. Data 13 de fevereiro de 2026